



Deep Learning Assignment

Assignment Deep Learning_assignment.ipynb run screenshot

BITS ID: 2026AJ05347

Step 1

The screenshot shows a Jupyter Notebook interface within Visual Studio Code. The browser address bar at the top displays a URL from g1.prayogshala.bits-pilani.ac.in. The VS Code window title is '2026aj05347_Assignment.executed.ipynb - workspace - Visual Studio Code'. The notebook's title bar indicates it is '2026aj05347_Assignment.executed.ipynb' and the current cell is 'Deep Neural Networks(AIMLG511) Assignment 01'. The notebook content includes the following text:

Deep Neural Networks(AIMLG511) Assignment 01

Comparing a Linear Model and a Multi-Layer Perceptron from Scratch

Student ID: 2026aj05347

Dataset: Breast Cancer Wisconsin (Diagnostic)

Problem: Binary classification

Baseline: Logistic Regression + Gradient Descent

Deep model: Multi-Layer Perceptron (MLP) + Backpropagation

This notebook follows the assignment sequence: dataset selection → preprocessing → baseline → MLP → evaluation → comparison → analysis.

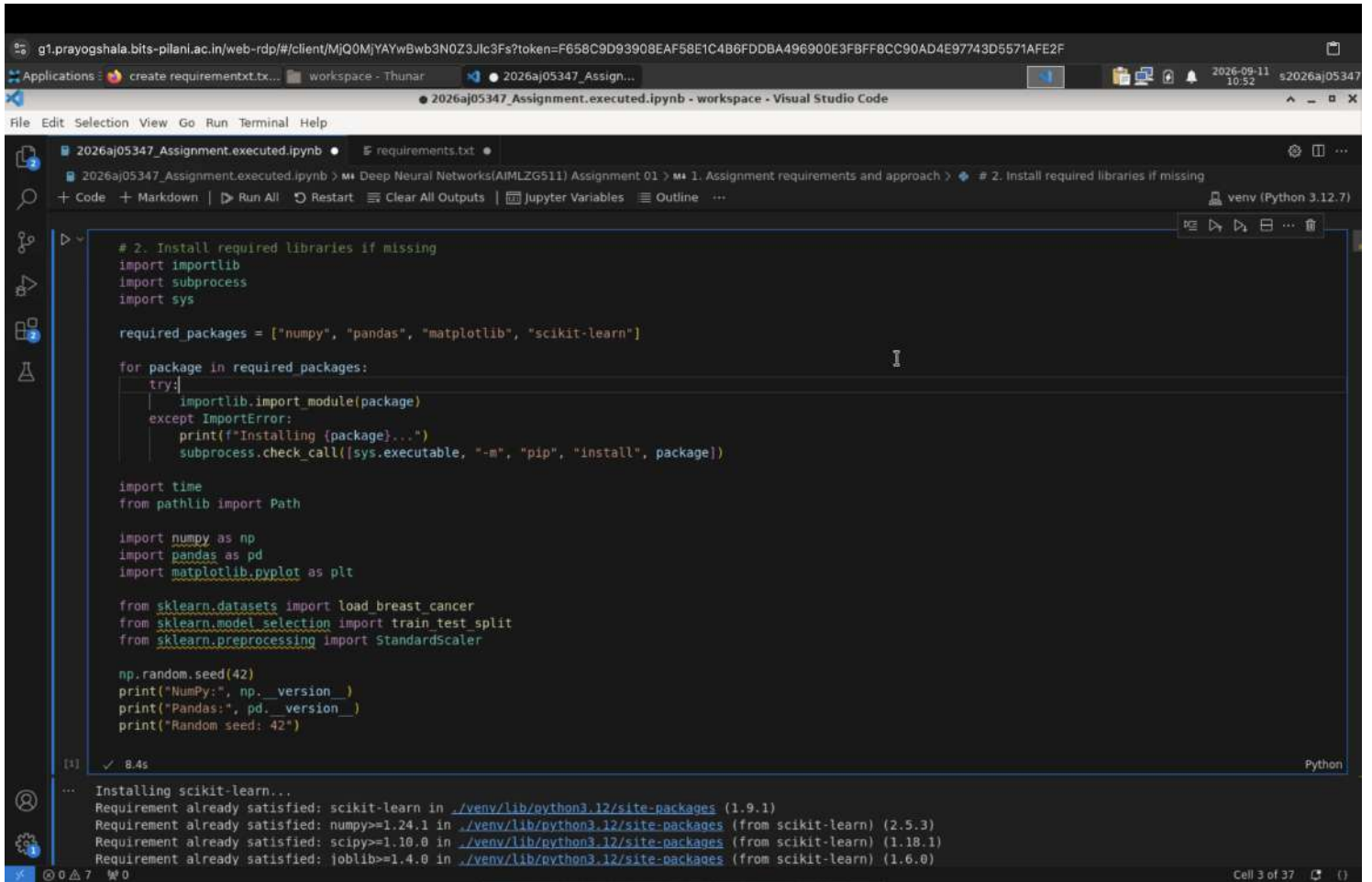
1. Assignment requirements and approach

The assignment requires a real-world dataset with at least 500 samples and 5 features, a baseline model implemented from scratch, an MLP implemented from scratch, test-set comparison, visualizations, and an analysis below 200 words.

For binary classification, the course notes use a weighted sum + sigmoid for the single-neuron baseline, Binary Cross-Entropy (BCE), gradient descent/SGD, ReLU hidden layers, a sigmoid output, and backpropagation.

The bottom status bar of VS Code shows 'Cell 1 of 37'.

Step 2



```
# 2. Install required libraries if missing
import importlib
import subprocess
import sys

required_packages = ["numpy", "pandas", "matplotlib", "scikit-learn"]

for package in required_packages:
    try:
        importlib.import_module(package)
    except ImportError:
        print(f"Installing {package}...")
        subprocess.check_call([sys.executable, "-m", "pip", "install", package])

import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Random seed: 42")
```

[1] ✓ 8.4s Python

... Installing scikit-learn...
Requirement already satisfied: scikit-learn in [./venv/lib/python3.12/site-packages](#) (1.9.1)
Requirement already satisfied: numpy<=1.24.1 in [./venv/lib/python3.12/site-packages](#) (from scikit-learn) (2.5.3)
Requirement already satisfied: scipy<=1.10.0 in [./venv/lib/python3.12/site-packages](#) (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib<=1.4.0 in [./venv/lib/python3.12/site-packages](#) (from scikit-learn) (1.6.0)

Step 3

The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is running in a Python 3.12.7 environment. The first cell shows the output of a requirements check, indicating that several libraries are already satisfied. The second cell shows the dataset schema, including feature names and columns. The third cell shows the output of a print statement indicating the number of model features is 30.

```
Requirement already satisfied: threadpoolctl>=3.5.0 in ./venv/lib/python3.12/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: cloudpickle>=3.0 in ./venv/lib/python3.12/site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)
NumPy: 2.5.3
Pandas: 3.0.5
Random seed: 42

# 3. Dataset schema
FEATURE_NAMES = ['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave_points_mean', 'symmetry_mean', 'fractal_dimension']
COLUMNS = ['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave_points_mean', 'symmetry_mean', 'fractal_dimension']

print("Number of model features:", len(FEATURE_NAMES))
```

[2] ✓ 0.0s Python

... Number of model features: 30

2. Load the dataset

The notebook first checks for `wdbc.data` beside the notebook. If it is not present, it tries the public UCI URL and then a raw GitHub mirror.

I selected the Breast Cancer Wisconsin (Diagnostic) dataset because it is a well-known real-world binary classification problem with 569 samples and 30 numerical diagnostic features, which satisfies the assignment requirements for dataset size and feature count. It also provides a clear medical interpretation: malignant cases are the positive class, so Recall is a suitable primary metric because missing a cancer case is more costly than a false alarm.

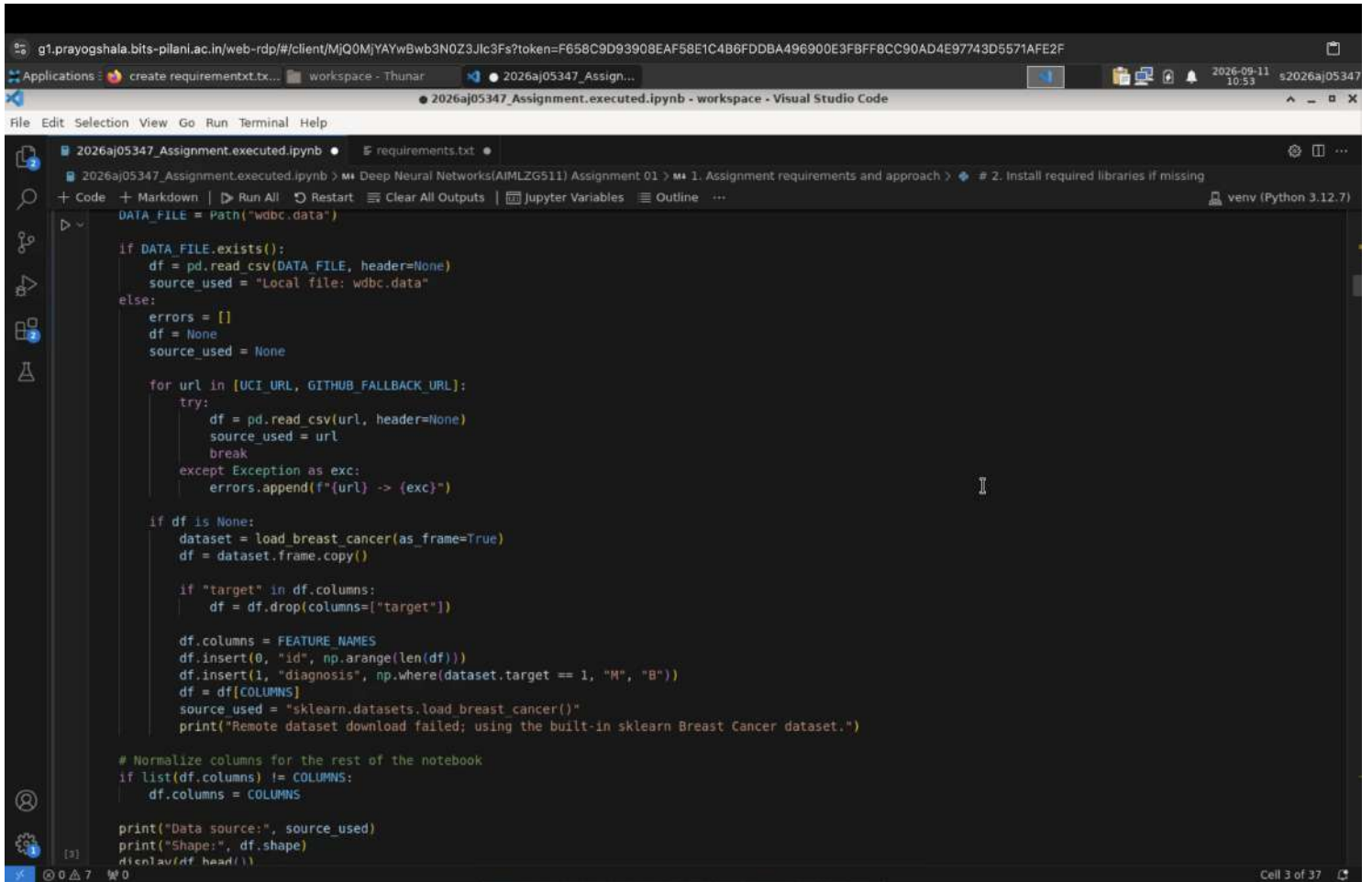
The actual model implementations do not use `sklearn` model classes.

```
# 4. Load WDBC data
UCI_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc.data"
GITHUB_FALLBACK_URL = (
    "https://raw.githubusercontent.com/rasbt/"
    "python-machine-learning-book/master/code/datasets/wdbc/wdbc.data"
```

[3]

Cell 3 of 37

Step 4



The screenshot shows a Visual Studio Code window with a Jupyter Notebook. The notebook is titled "2026aj05347_Assignment.executed.ipynb" and is part of a workspace named "workspace - Thunar". The code in the cell is as follows:

```
DATA_FILE = Path("wdbc.data")

if DATA_FILE.exists():
    df = pd.read_csv(DATA_FILE, header=None)
    source_used = "Local file: wdbc.data"
else:
    errors = []
    df = None
    source_used = None

    for url in [UCI_URL, GITHUB_FALLBACK_URL]:
        try:
            df = pd.read_csv(url, header=None)
            source_used = url
            break
        except Exception as exc:
            errors.append(f"{url} -> {exc}")

    if df is None:
        dataset = load_breast_cancer(as_frame=True)
        df = dataset.frame.copy()

        if "target" in df.columns:
            df = df.drop(columns=["target"])

        df.columns = FEATURE_NAMES
        df.insert(0, "id", np.arange(len(df)))
        df.insert(1, "diagnosis", np.where(dataset.target == 1, "M", "B"))
        df = df[COLUMNS]
        source_used = "sklearn.datasets.load_breast_cancer()"
        print("Remote dataset download failed; using the built-in sklearn Breast Cancer dataset.")

# Normalize columns for the rest of the notebook
if list(df.columns) != COLUMNS:
    df.columns = COLUMNS

print("Data source:", source_used)
print("Shape:", df.shape)
display(df.head())
```

The code attempts to load a local CSV file "wdbc.data". If it fails, it tries to download it from UCI or GitHub. If that also fails, it uses the built-in sklearn Breast Cancer dataset. Finally, it normalizes the columns to match the expected COLUMNS list and prints the data source, shape, and a head of the dataframe.

Step 5

g1.prayogshala.bits-pilani.ac.in/web-rdp/#/client/MjQOMjYAYwBwb3N0Z3Jlc3Fs?token=F658C9D93908EAF58E1C4B6FDDBA496900E3FBFF8CC90AD4E97743D5571AFE2F

Applications: create requirement.txt... workspace - Thunar 2026aj05347_Assign...

2026aj05347_Assignment.executed.ipynb - workspace - Visual Studio Code

File Edit Selection View Go Run Terminal Help

2026aj05347_Assignment.executed.ipynb requirements.txt

2026aj05347_Assignment.executed.ipynb > Deep Neural Networks(AIIMLZG511) Assignment 01 > 1. Assignment requirements and approach > 2. Install required libraries if missing

+ Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline ...

venv (Python 3.12.7) mynon

... Data source: https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc_data
Shape: (569, 32)

...

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave_points_mean	...
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...

5 rows x 32 columns

+ Code + Markdown

```
# 5. Data quality checks
print("Missing values:", int(df.isna().sum().sum()))
print("Duplicate rows:", int(df.duplicated().sum()))

print("\nClass distribution:")
display(df["diagnosis"].value_counts())

print("Feature count:", len(FEATURE_NAMES))
print("Sample count:", len(df))

assert len(df) >= 500
assert len(FEATURE_NAMES) >= 5
```

[4] ✓ 0.0s Python

... Missing values: 0
Duplicate rows: 0

Class distribution:

... diagnosis

0 7 W 0

Cell 3 of 37

Step 6

The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is running on a remote server, as indicated by the URL in the browser window: `g1.prayogshala.bits-pilani.ac.in/web-rdp/#/client/MjQ0MjYAYwBwb3N0Z3Jlc3Fs?token=F658C9D93908EAF58E1C4B6FDDBA496900E3FBFF8CC90AD4E97743D5571AFE2F`. The notebook is open to a cell titled "Dataset description".

The cell contains the following text:

M (malignant) is the positive class (1) and B (benign) is the negative class (0). The id field is an identifier, so it is excluded from the model. The remaining 30 columns are numeric diagnostic measurements.

Primary metric: Recall, because the positive class represents malignancy and the course notes emphasize recall when false negatives are costly.

The cell also contains the following code:

```
# 6. Build X and y
X = df[FEATURE_NAMES].astype(float).to_numpy()
y = df["diagnosis"].eq("M").astype(int).to_numpy().reshape(-1, 1)

print("X shape:", X.shape)
print("y shape:", y.shape)
print("Positive class rate:", float(y.mean()))
```

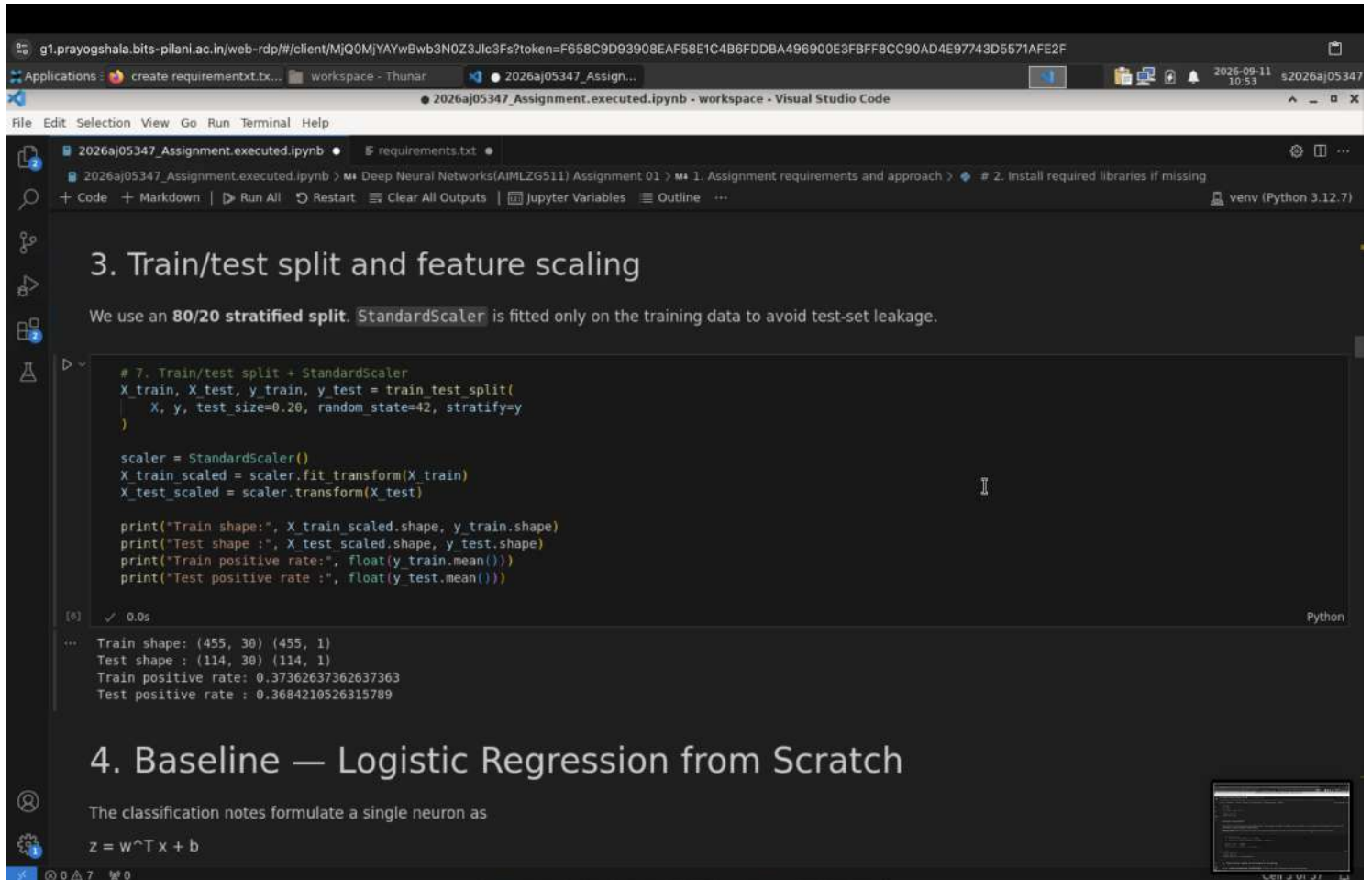
The output of the code is:

```
X shape: (569, 30)
y shape: (569, 1)
Positive class rate: 0.37258347978910367
```

The notebook is currently on cell 3 of 37, titled "3. Train/test split and feature scaling". The text in the cell is:

We use an **80/20 stratified split**. `StandardScaler` is fitted only on the training data to avoid test-set leakage.

Step 7



g1.prayogshala.bits-pilani.ac.in/web-rdp/#/client/MjQ0MjYAYwBwb3N0Z3Jlc3Fs?token=F658C9D93908EAF58E1C4B6FDDBA496900E3FBFF8CC90AD4E97743D5571AFE2F

Applications: create requirement.txt.tx... workspace - Thunar 2026aj05347_Assign...

2026aj05347_Assignment.executed.ipynb - workspace - Visual Studio Code

File Edit Selection View Go Run Terminal Help

2026aj05347_Assignment.executed.ipynb • requirements.txt •

2026aj05347_Assignment.executed.ipynb > Deep Neural Networks(AIMLZG511) Assignment 01 > 1. Assignment requirements and approach > 2. Install required libraries if missing

+ Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.7)

3. Train/test split and feature scaling

We use an **80/20 stratified split**. `StandardScaler` is fitted only on the training data to avoid test-set leakage.

```
# 7. Train/test split + StandardScaler
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Train shape:", X_train_scaled.shape, y_train.shape)
print("Test shape :", X_test_scaled.shape, y_test.shape)
print("Train positive rate:", float(y_train.mean()))
print("Test positive rate :", float(y_test.mean()))
```

[e] ✓ 0.0s Python

```
... Train shape: (455, 30) (455, 1)
Test shape : (114, 30) (114, 1)
Train positive rate: 0.37362637362637363
Test positive rate : 0.3684210526315789
```

4. Baseline — Logistic Regression from Scratch

The classification notes formulate a single neuron as

$$z = w^T x + b$$

Step 8

The screenshot shows a Visual Studio Code editor window with a Jupyter Notebook. The browser address bar at the top displays a URL from g1.prayogshala.bits-pilani.ac.in. The VS Code interface includes a menu bar (File, Edit, Selection, View, Go, Run, Terminal, Help) and a toolbar with icons for file operations and execution. The notebook has two tabs: '2026aj05347_Assignment.executed.ipynb' and 'requirements.txt'. The active cell in the notebook contains the following text:

4. Baseline — Logistic Regression from Scratch

The classification notes formulate a single neuron as

$$z = w^T x + b$$

followed by sigmoid:

$$\hat{y} = 1 / (1 + \exp(-z))$$

For binary classification we minimize Binary Cross-Entropy:

$$J = -(1/N) \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

The gradient has the simple form:

$$\nabla J = (1/N) X^T(\hat{y} - y)$$

and gradient descent is:

$$w \leftarrow w - \eta \nabla J$$

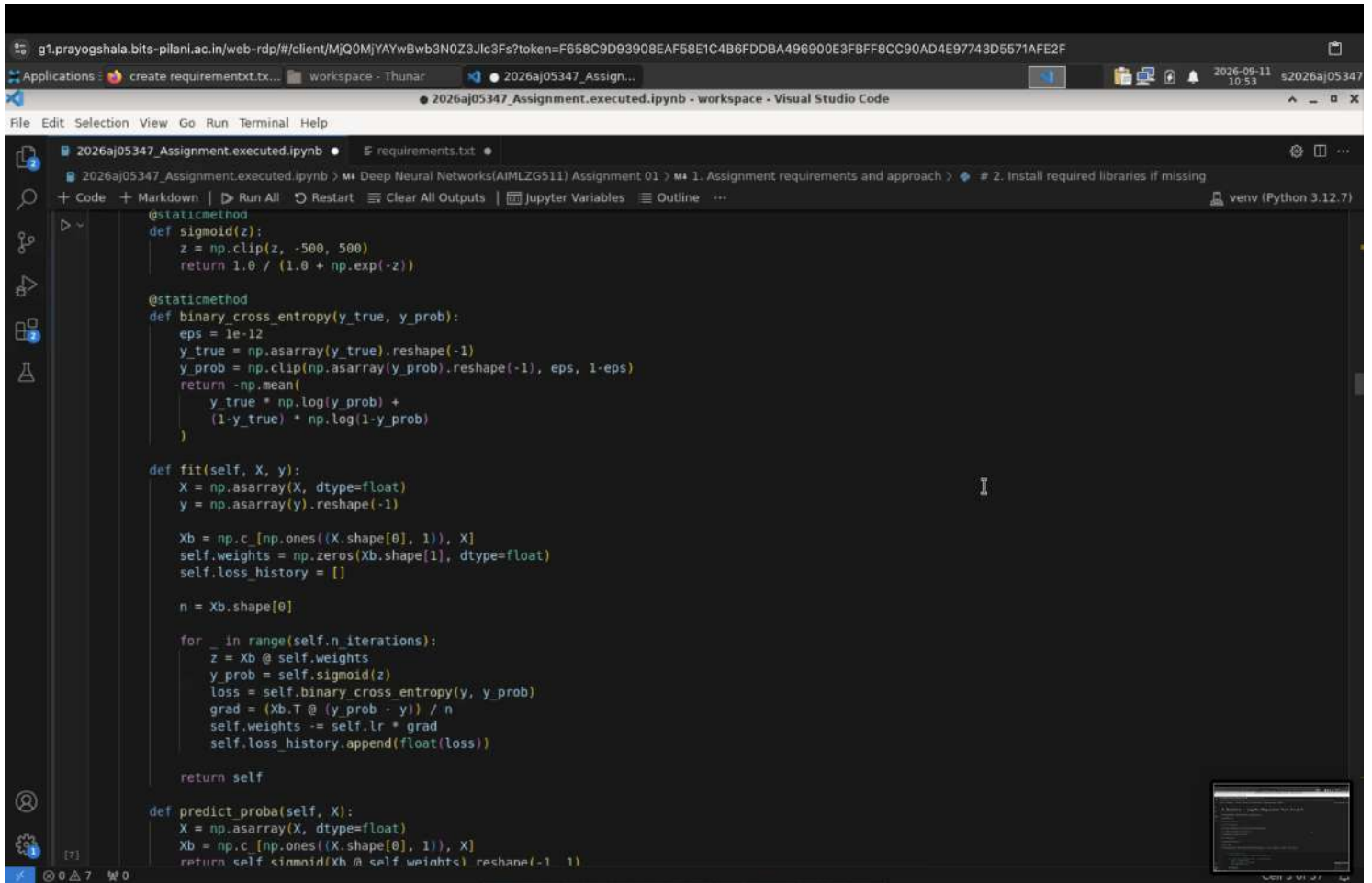
The implementation below explicitly follows forward pass → loss → gradient → update → loss history.

```
class LogisticRegressionGD:
    """Binary logistic regression implemented with NumPy only."""

    def __init__(self, learning_rate=0.1, n_iterations=1500):
        self.lr = learning_rate
        self.n_iterations = n_iterations
        self.loss_history = []

    @staticmethod
    def sigmoid(z):
```

Step 9



The screenshot shows a Visual Studio Code window with a Jupyter Notebook. The notebook is titled "2026aj05347_Assignment.executed.ipynb" and is part of a workspace named "workspace - Thunar". The code is written in Python and defines a neural network class with the following methods:

```
@staticmethod
def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

@staticmethod
def binary_cross_entropy(y_true, y_prob):
    eps = 1e-12
    y_true = np.asarray(y_true).reshape(-1)
    y_prob = np.clip(np.asarray(y_prob).reshape(-1), eps, 1-eps)
    return -np.mean(
        y_true * np.log(y_prob) +
        (1-y_true) * np.log(1-y_prob)
    )

def fit(self, X, y):
    X = np.asarray(X, dtype=float)
    y = np.asarray(y).reshape(-1)

    Xb = np.c_[np.ones((X.shape[0], 1)), X]
    self.weights = np.zeros(Xb.shape[1], dtype=float)
    self.loss_history = []

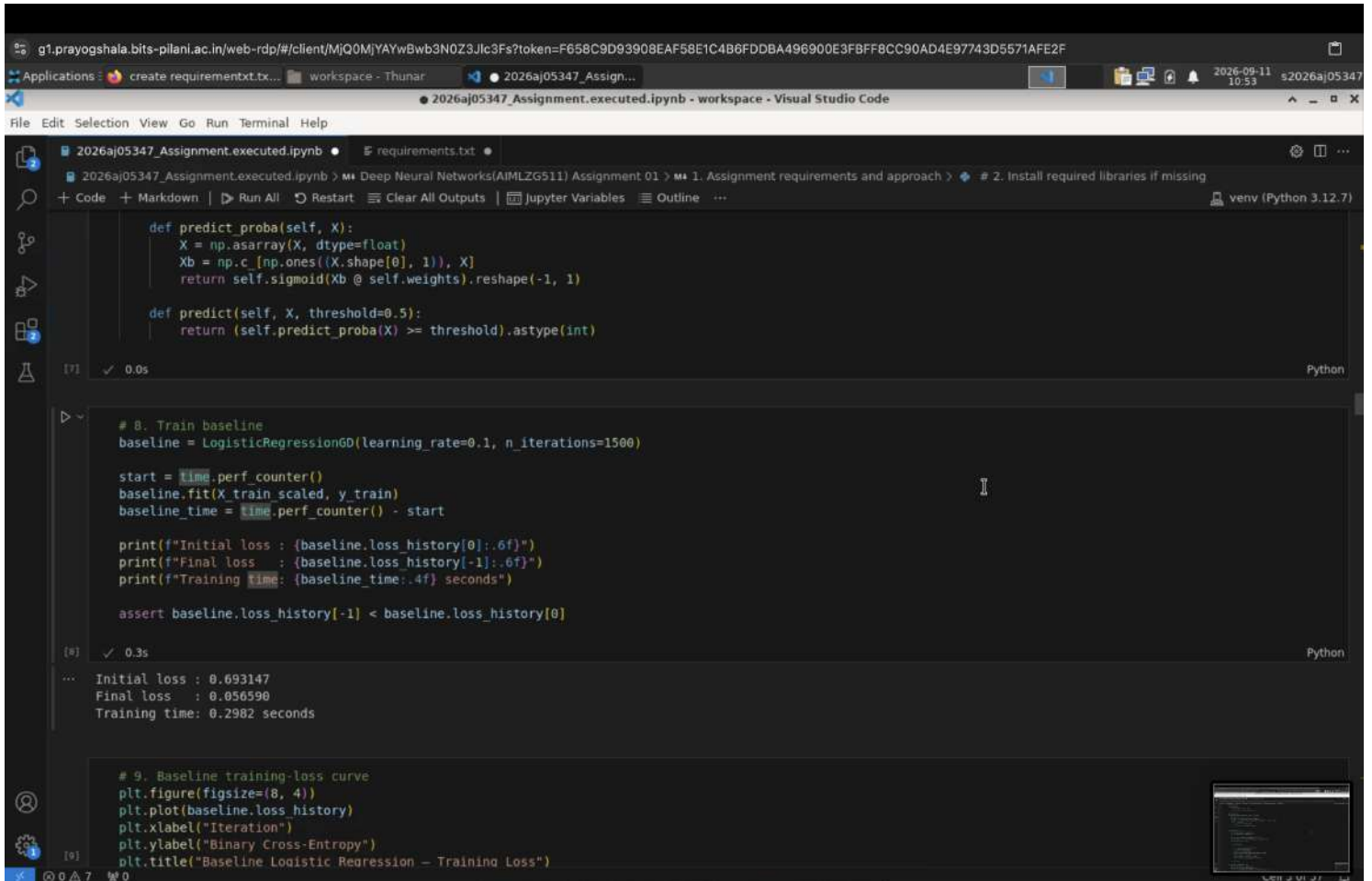
    n = Xb.shape[0]

    for _ in range(self.n_iterations):
        z = Xb @ self.weights
        y_prob = self.sigmoid(z)
        loss = self.binary_cross_entropy(y, y_prob)
        grad = (Xb.T @ (y_prob - y)) / n
        self.weights -= self.lr * grad
        self.loss_history.append(float(loss))

    return self

def predict_proba(self, X):
    X = np.asarray(X, dtype=float)
    Xb = np.c_[np.ones((X.shape[0], 1)), X]
    return self.sigmoid(Xb @ self.weights).reshape(-1, 1)
```

Step 10



```
g1.prayogshala.bits-pilani.ac.in/web-rdp/#/client/MjQ0MjYAYwBwb3N0Z3Jlc3Fs?token=F658C9D93908EAF58E1C4B6FDDBA496900E3FBFF8CC90AD4E97743D5571AFE2F
Applications: create requirementstxt.tx... workspace - Thunar 2026aj05347_Assign... 2026-09-11 10:53 s2026aj05347
2026aj05347_Assignment.executed.ipynb - workspace - Visual Studio Code
File Edit Selection View Go Run Terminal Help
2026aj05347_Assignment.executed.ipynb requirements.txt
2026aj05347_Assignment.executed.ipynb > Deep Neural Networks(AIMLG511) Assignment 01 > 1. Assignment requirements and approach > 2. Install required libraries if missing
+ Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline ... venv (Python 3.12.7)

def predict_proba(self, X):
    X = np.asarray(X, dtype=float)
    Xb = np.c_[np.ones((X.shape[0], 1)), X]
    return self.sigmoid(Xb @ self.weights).reshape(-1, 1)

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X) >= threshold).astype(int)

[7] ✓ 0.0s Python

# 8. Train baseline
baseline = LogisticRegressionGD(learning_rate=0.1, n_iterations=1500)

start = time.perf_counter()
baseline.fit(X_train_scaled, y_train)
baseline_time = time.perf_counter() - start

print(f"Initial loss : {baseline.loss_history[0]:.6f}")
print(f"Final loss : {baseline.loss_history[-1]:.6f}")
print(f"Training time: {baseline_time:.4f} seconds")

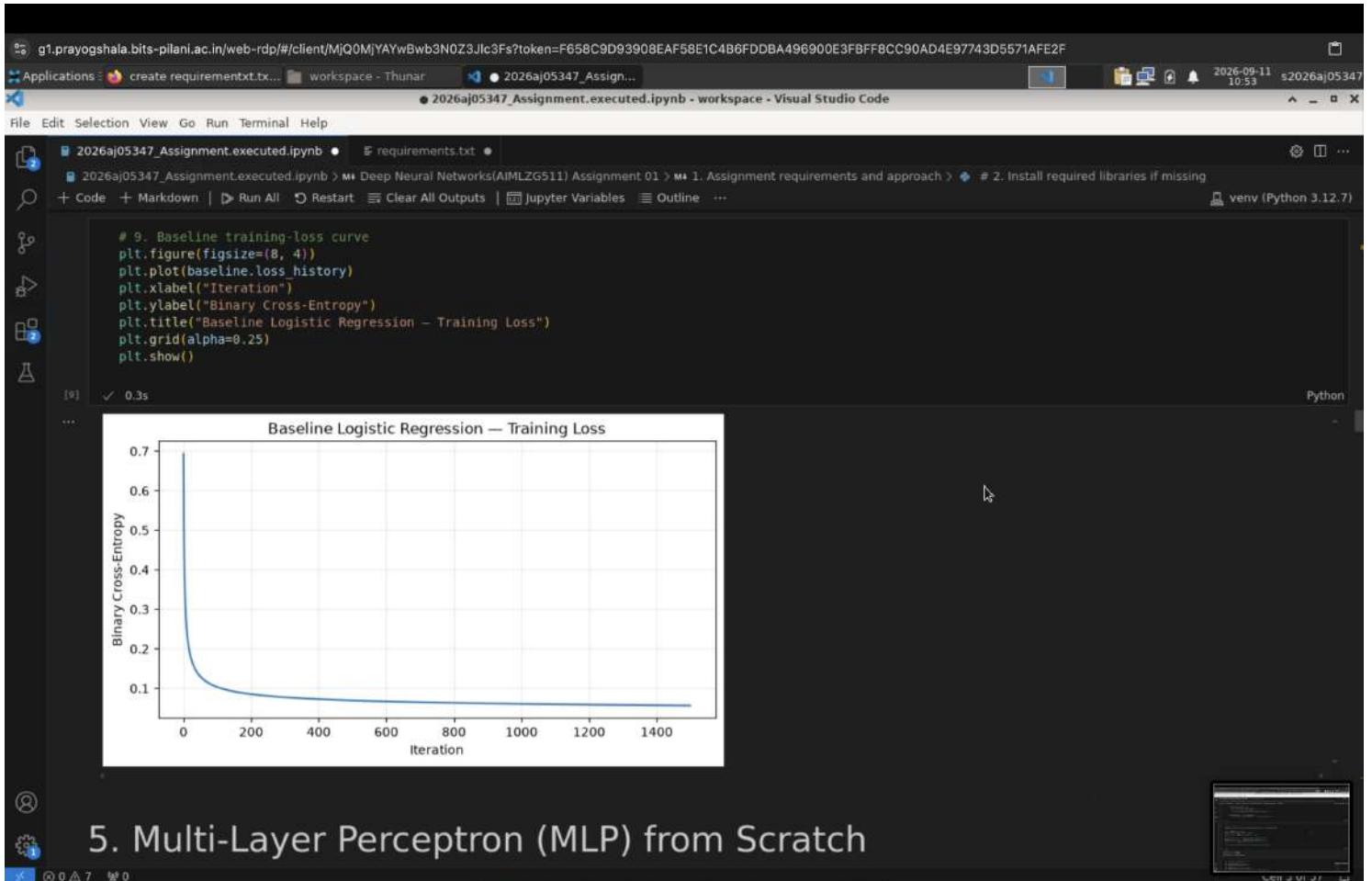
assert baseline.loss_history[-1] < baseline.loss_history[0]

[8] ✓ 0.3s Python
... Initial loss : 0.693147
Final loss : 0.056590
Training time: 0.2982 seconds

# 9. Baseline training-loss curve
plt.figure(figsize=(8, 4))
plt.plot(baseline.loss_history)
plt.xlabel("Iteration")
plt.ylabel("Binary Cross-Entropy")
plt.title("Baseline Logistic Regression - Training Loss")

[9] ✓ 0.0s Python
```

Step 11



Step 12

The screenshot shows a Jupyter Notebook titled "5. Multi-Layer Perceptron (MLP) from Scratch" in a Visual Studio Code environment. The notebook is part of a workspace named "2026aj05347_Assignment.executed.ipynb". The browser address bar shows a URL from g1.prayogshala.bits-pilani.ac.in. The notebook content includes handwritten notes and code for implementing an MLP.

5. Multi-Layer Perceptron (MLP) from Scratch

The handwritten notes show the weighted-sum formulation and the limitation of a single perceptron on XOR. The DFNN notes extend the idea by stacking layers with non-linear activations.

Architecture: 30 → 16 → 8 → 1

- Hidden layers: ReLU
- Output: Sigmoid
- Loss: BCE
- Optimizer: mini-batch SGD
- Backpropagation: chain rule
- Weight initialization: He-style for ReLU layers

For BCE + sigmoid, the output-layer error signal simplifies to:

$$\delta(L) = \hat{Y} - Y$$

For a hidden layer:

$$\delta(l) = (\delta(l+1) W(l+1)^T) \odot \text{ReLU}'(Z(l))$$

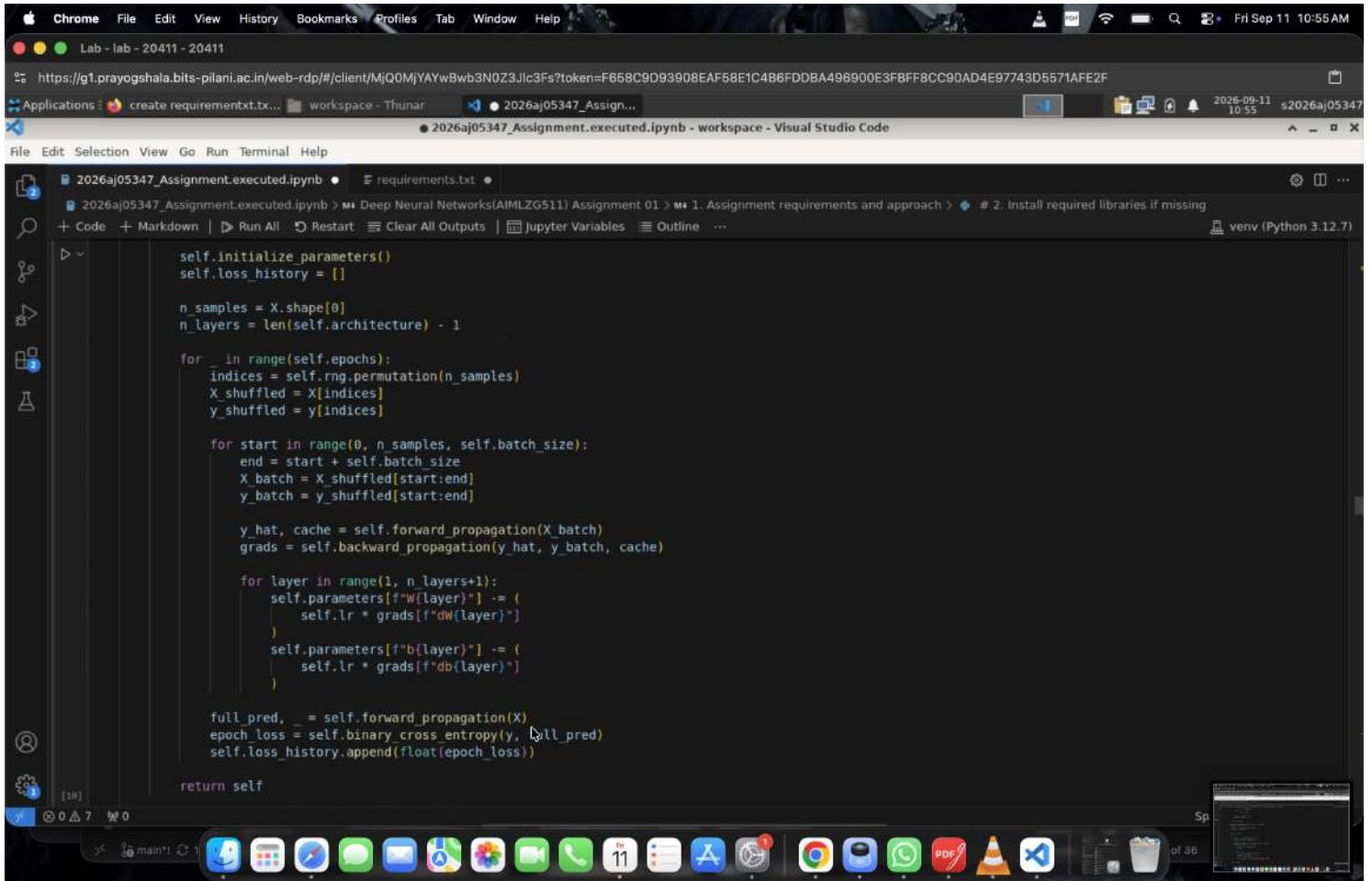
Then:

$$dW(l) = (1/B) H(l-1)^T \delta(l)$$
$$db(l) = \text{mean}(\delta(l))$$

Finally, every parameter is updated with gradient descent.

```
class MLP:
    """Fully connected MLP implemented from scratch with NumPy."""
    def __init__(
```

Step 16



```
self.initialize_parameters()
self.loss_history = []

n_samples = X.shape[0]
n_layers = len(self.architecture) - 1

for _ in range(self.epochs):
    indices = self.rng.permutation(n_samples)
    X_shuffled = X[indices]
    y_shuffled = y[indices]

    for start in range(0, n_samples, self.batch_size):
        end = start + self.batch_size
        X_batch = X_shuffled[start:end]
        y_batch = y_shuffled[start:end]

        y_hat, cache = self.forward_propagation(X_batch)
        grads = self.backward_propagation(y_hat, y_batch, cache)

        for layer in range(1, n_layers+1):
            self.parameters[f"w{layer}"] -= (
                self.lr * grads[f"dW{layer}"]
            )
            self.parameters[f"b{layer}"] -= (
                self.lr * grads[f"db{layer}"]
            )

    full_pred, _ = self.forward_propagation(X)
    epoch_loss = self.binary_cross_entropy(y, full_pred)
    self.loss_history.append(float(epoch_loss))

return self
```

Step 17

The screenshot shows a Visual Studio Code editor window with a Jupyter Notebook open. The notebook is titled "2026aj05347_Assignment.executed.ipynb" and is part of a workspace named "workspace - Thunar". The code is written in Python and defines an MLP class with methods for forward propagation, prediction, and counting parameters. The notebook is running on a Python 3.12.7 environment.

```
epoch_loss = self.binary_cross_entropy(y, full_pred)
self.loss_history.append(float(epoch_loss))

return self

def predict_proba(self, X):
    y_hat, _ = self.forward_propagation(np.asarray(X, dtype=float))
    return y_hat

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X) >= threshold).astype(int)

def count_parameters(self):
    total = 0
    for layer in range(1, len(self.architecture)):
        total += np.prod(self.parameters[f"w{layer}"].shape)
        total += np.prod(self.parameters[f"b{layer}"].shape)
    return int(total)

# 10. Train MLP
mlp = MLP(
    architecture=[30, 16, 8, 1],
    learning_rate=0.01,
    epochs=250,
    batch_size=32,
    random_state=42,
)

start = time.perf_counter()
mlp.fit(X_train_scaled, y_train)
```

The bottom of the image shows the macOS dock with various application icons, including Finder, Safari, Mail, Messages, Photos, App Store, and others. The system status bar at the top right indicates the date and time as "Fri Sep 11 10:55 AM".

Step 18

The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is running on a remote server, as indicated by the URL in the browser window: <https://g1.prayogshala.bits-pilani.ac.in/web-rdp/#/client/MjQ0MjYAYwBwb3N0Z3Jlc3Fs?token=F658C9D93908EAF58E1C4B6FDDBA496900E3FBFF8CC90AD4E97743D5671AFE2F>. The notebook is open to a cell containing Python code for training an MLP model.

```
start = time.perf_counter()
mlp.fit(X_train_scaled, y_train)
mlp_time = time.perf_counter() - start

print("Architecture:", mlp.architecture)
print("Trainable parameters:", f"{mlp.count_parameters():,}")
print(f"Initial loss : {mlp.loss_history[0]:.6f}")
print(f"Final loss : {mlp.loss_history[-1]:.6f}")
print(f"Training time: {mlp_time:.4f} seconds")

assert mlp.loss_history[-1] < mlp.loss_history[0]
```

The output of the code shows the following results:

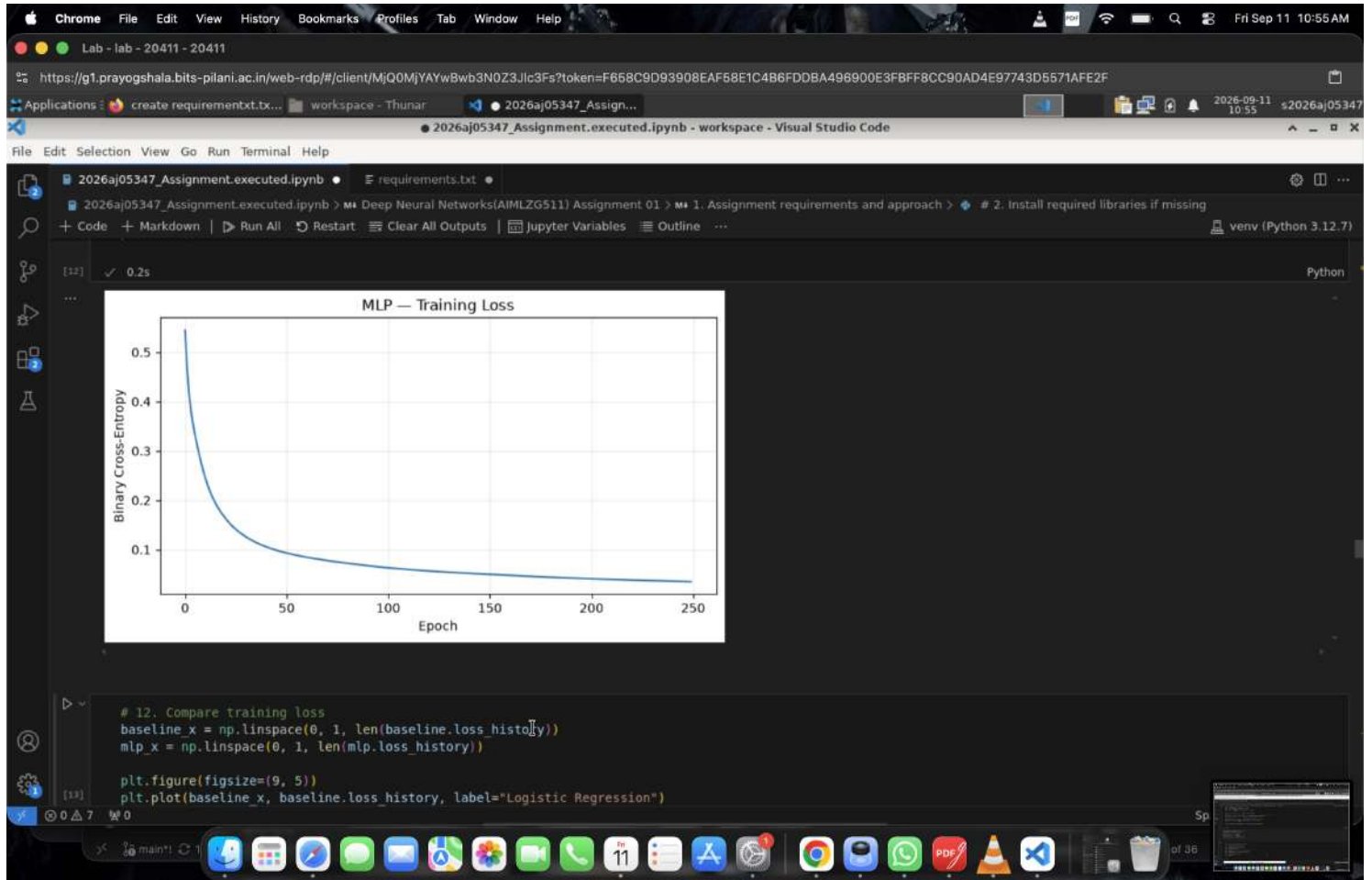
```
[11] ✓ 1.0s
... Architecture: [30, 16, 8, 1]
Trainable parameters: 641
Initial loss : 0.544427
Final loss : 0.034834
Training time: 1.0288 seconds
```

Below the output, there is a code cell for plotting the training loss curve:

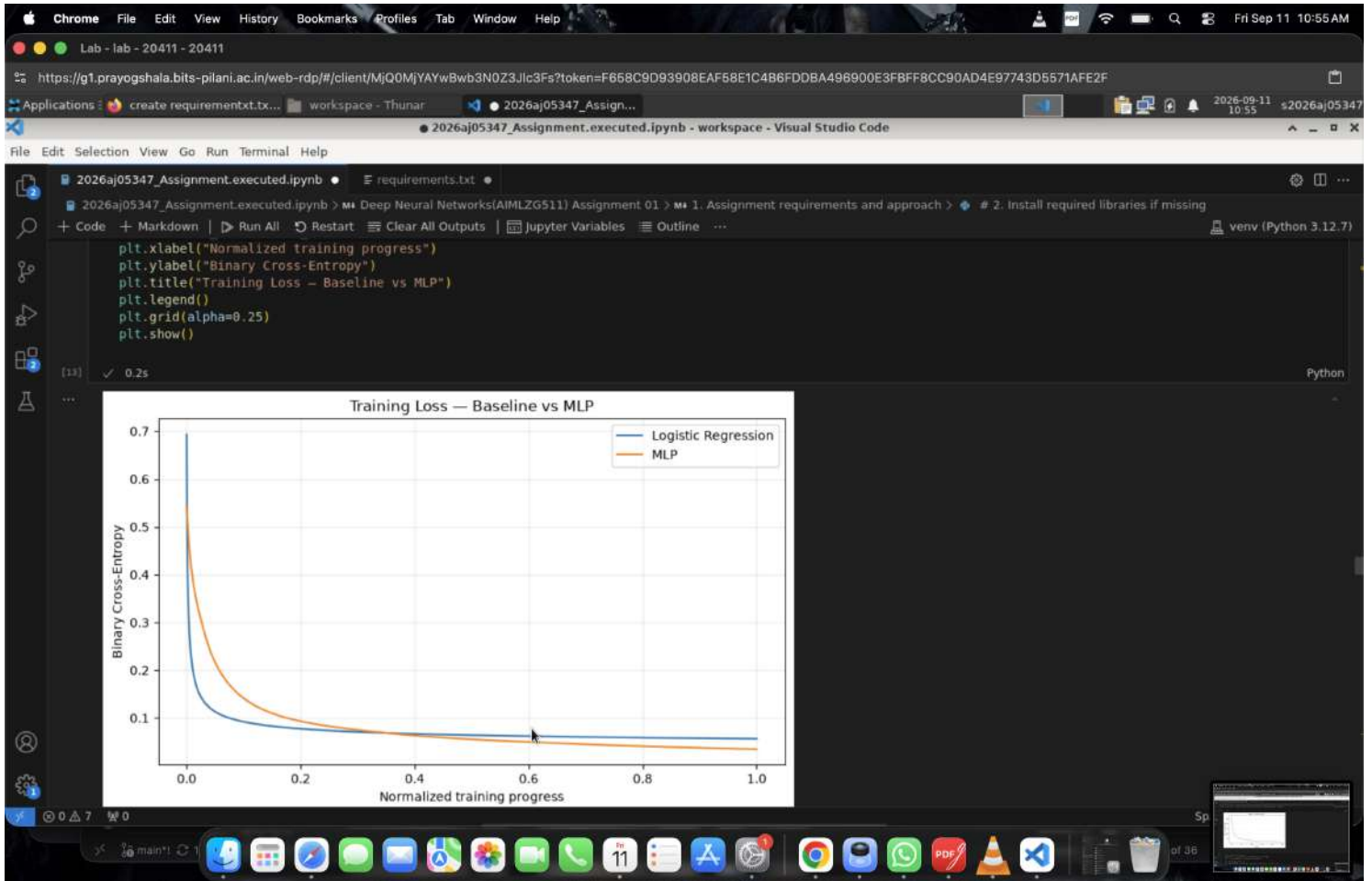
```
# 11. MLP training-loss curve
plt.figure(figsize=(8, 4))
plt.plot(mlp.loss_history)
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy")
plt.title("MLP - Training Loss")
plt.grid(alpha=0.25)
plt.show()
```

The output of this cell shows a plot titled "MLP — Training Loss". The plot displays the training loss (Binary Cross-Entropy) over epochs, showing a decreasing trend. The plot is displayed in a separate window titled "MLP — Training Loss".

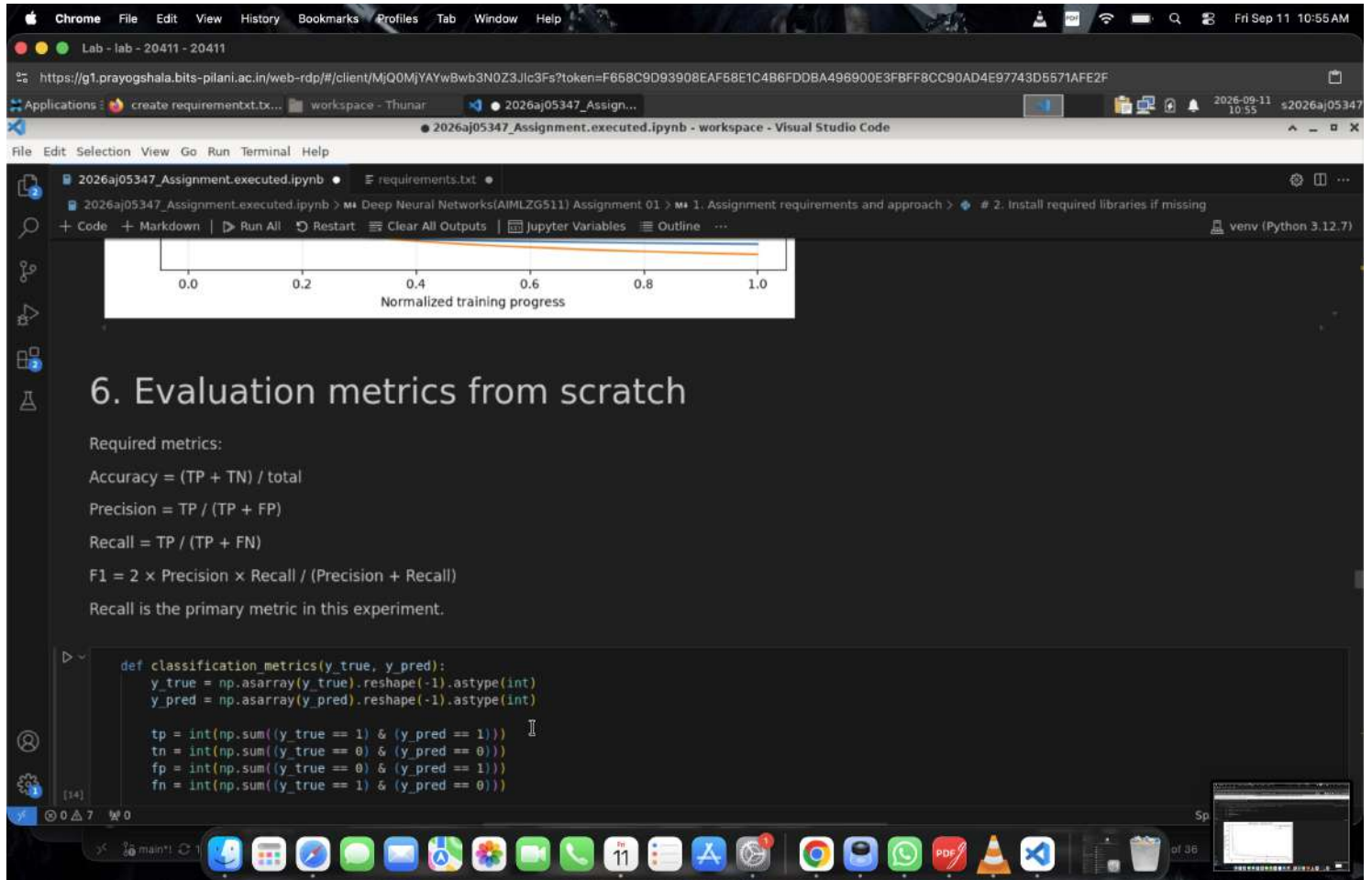
Step 19



Step 20



Step 21



The screenshot shows a Jupyter Notebook interface within Visual Studio Code. At the top, a progress bar indicates 'Normalized training progress' from 0.0 to 1.0. Below the progress bar, the notebook content is displayed. The title of the notebook is '2026aj05347_Assignment.executed.ipynb'. The current cell is titled '6. Evaluation metrics from scratch'. The cell contains the following text:

Required metrics:

Accuracy = $(TP + TN) / \text{total}$

Precision = $TP / (TP + FP)$

Recall = $TP / (TP + FN)$

$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

Recall is the primary metric in this experiment.

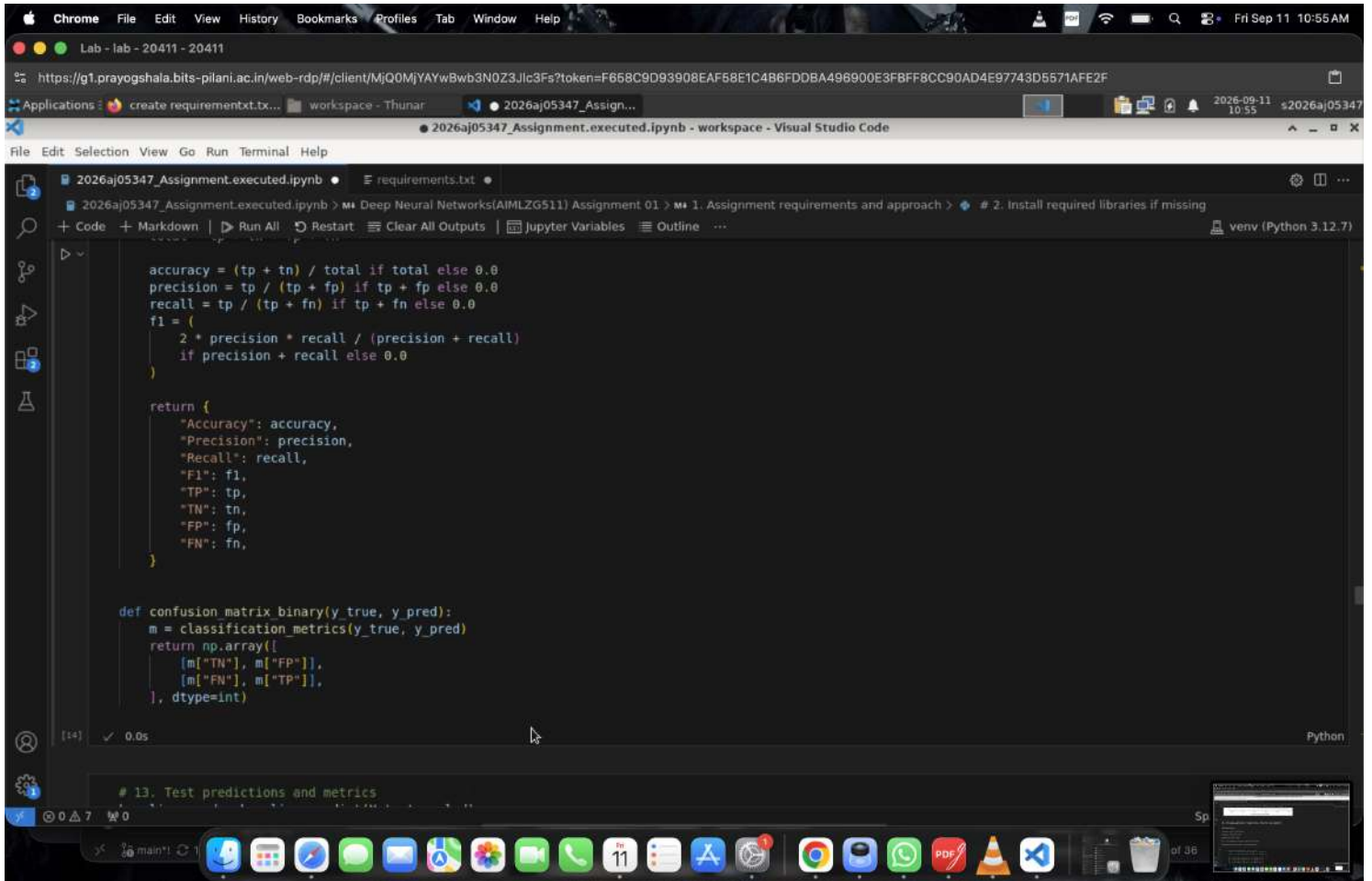
Below the text, a code cell contains the following Python code:

```
def classification_metrics(y_true, y_pred):
    y_true = np.asarray(y_true).reshape(-1).astype(int)
    y_pred = np.asarray(y_pred).reshape(-1).astype(int)

    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))
```

The code cell is currently selected, and the cursor is positioned at the end of the last line. The bottom of the screen shows the macOS dock with various application icons.

Step 22



The screenshot shows a web-based IDE interface with a browser window at the top and a code editor below. The browser window displays a URL from a remote server. The code editor shows a Python file named '2026aj05347_Assignment.executed.ipynb' with the following code:

```
accuracy = (tp + tn) / total if total else 0.0
precision = tp / (tp + fp) if tp + fp else 0.0
recall = tp / (tp + fn) if tp + fn else 0.0
f1 = (
    2 * precision * recall / (precision + recall)
    if precision + recall else 0.0
)

return {
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1": f1,
    "TP": tp,
    "TN": tn,
    "FP": fp,
    "FN": fn,
}
```

```
def confusion_matrix_binary(y_true, y_pred):
    m = classification_metrics(y_true, y_pred)
    return np.array([
        [m["TN"], m["FP"]],
        [m["FN"], m["TP"]],
    ], dtype=int)
```

The interface includes a file explorer on the left, a terminal at the bottom, and a macOS dock at the very bottom with various application icons.

Step 23

The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is running on a remote server, as indicated by the URL in the browser window. The code is written in Python and is part of a series of steps for a Deep Neural Networks assignment.

The code in the notebook includes:

```
[14] ✓ 0.0s  
  
# 13. Test predictions and metrics  
baseline_pred = baseline.predict(X_test_scaled)  
mlp_pred = mlp.predict(X_test_scaled)  
  
baseline_metrics = classification_metrics(y_test, baseline_pred)  
mlp_metrics = classification_metrics(y_test, mlp_pred)  
  
metrics_df = pd.DataFrame(  
    [baseline_metrics, mlp_metrics],  
    index=["Logistic Regression", "MLP"],  
    columns=["Accuracy", "Precision", "Recall", "F1"]  
)  
#display(metrics_df.style.format("{:.4f}"))  
display(metrics_df.round(4))  
  
[15] ✓ 0.0s  
...  


|                     | Accuracy | Precision | Recall | F1     |
|---------------------|----------|-----------|--------|--------|
| Logistic Regression | 0.9737   | 0.9756    | 0.9524 | 0.9639 |
| MLP                 | 0.9737   | 1.0000    | 0.9286 | 0.9630 |

  
# 14. Confusion matrices  
fig, axes = plt.subplots(1, 2, figsize=(10, 4))  
  
[16] for ax, title, preds in [  
    (axes[0], "Logistic Regression", baseline_pred),
```

The output of the code shows a table of classification metrics for Logistic Regression and MLP models. The metrics are Accuracy, Precision, Recall, and F1 score, all rounded to 4 decimal places.

	Accuracy	Precision	Recall	F1
Logistic Regression	0.9737	0.9756	0.9524	0.9639
MLP	0.9737	1.0000	0.9286	0.9630

The notebook also includes a section for confusion matrices, which is partially visible in the screenshot.

Step 24

The screenshot shows a Jupyter Notebook interface within Visual Studio Code. The notebook is titled "2026aj05347_Assignment.executed.ipynb" and is part of a workspace named "workspace - Thunar". The code in the notebook is as follows:

```
# 14. Confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

for ax, title, preds in [
    (axes[0], "Logistic Regression", baseline_pred),
    (axes[1], "MLP", mlp_pred),
]:
    cm = confusion_matrix_binary(y_test, preds)

    ax.imshow(cm)
    ax.set_xticks([0, 1], ["Pred 0", "Pred 1"])
    ax.set_yticks([0, 1], ["Actual 0", "Actual 1"])
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_title(title)

    for i in range(2):
        for j in range(2):
            ax.text(j, i, str(cm[i, j]), ha="center", va="center")

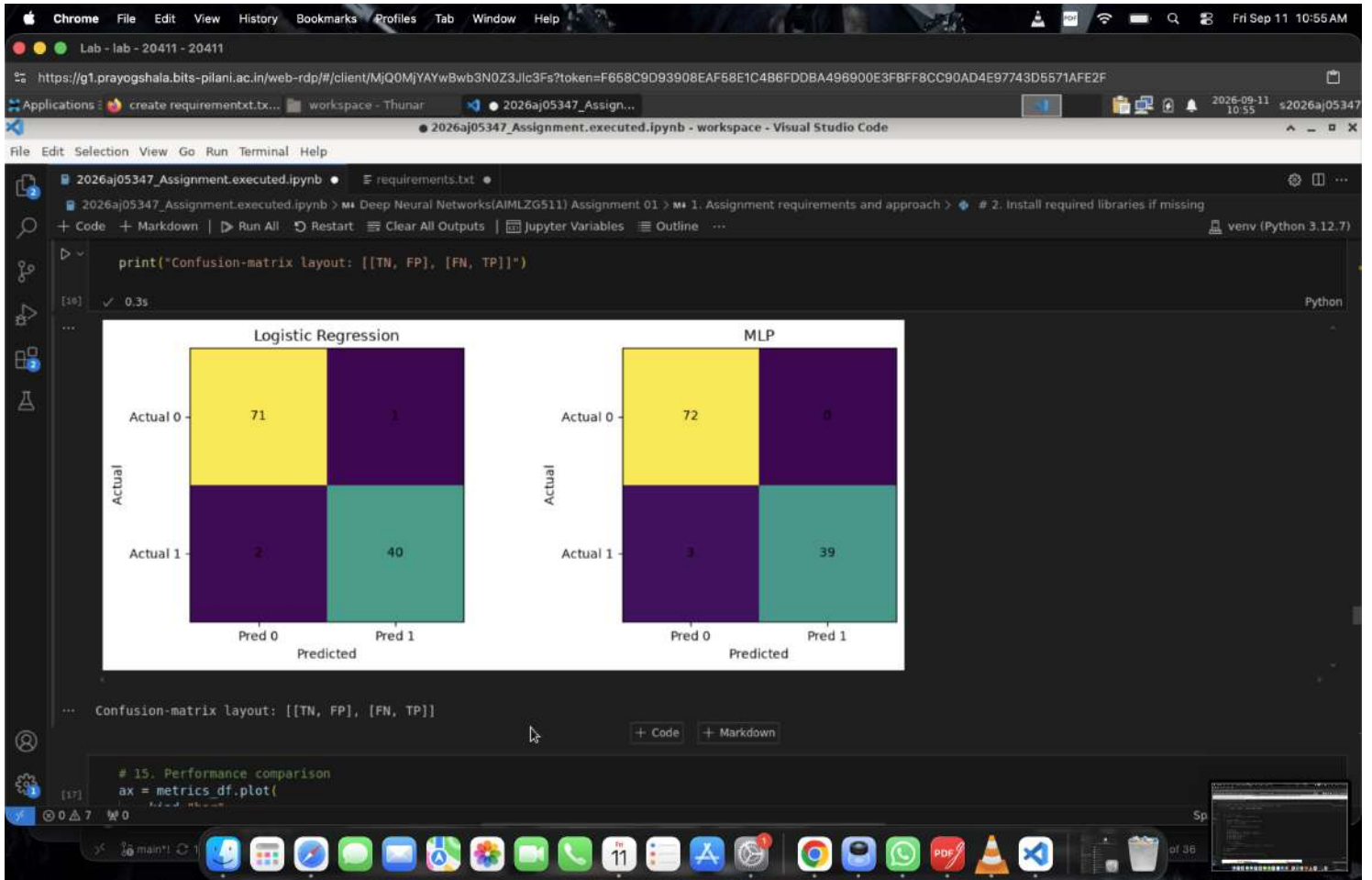
plt.tight_layout()
plt.show()

print("Confusion-matrix layout: [[TN, FP], [FN, TP]])")
```

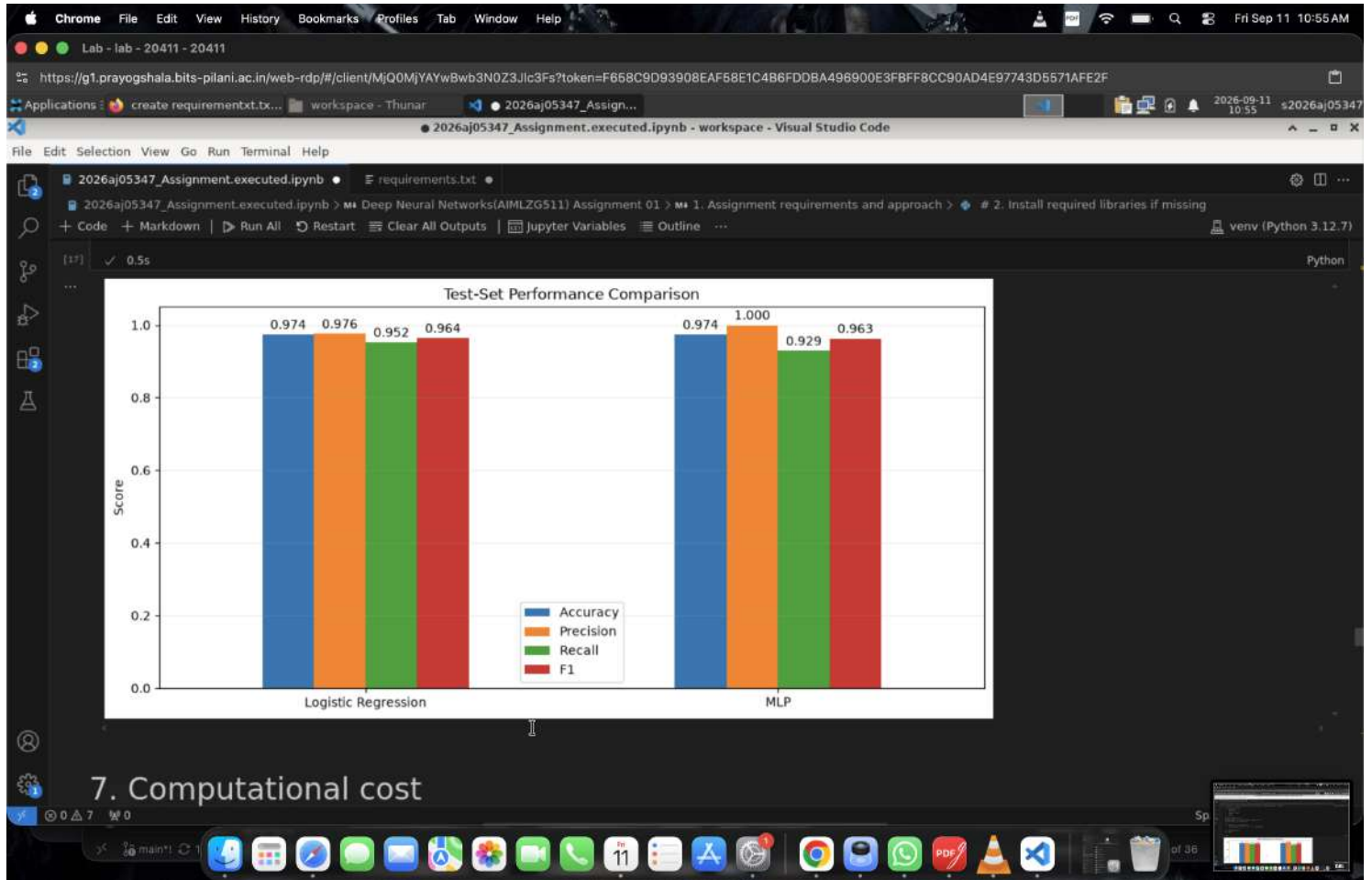
Below the code, a progress bar is visible, showing the progress of the "Logistic Regression" and "MLP" tasks. The "Logistic Regression" task is currently active and shows a progress of 0.3s. The "MLP" task is also shown but is not currently active.

The bottom of the screen shows the macOS dock with various application icons, including Finder, Safari, Mail, Messages, Photos, App Store, and others.

Step 25



Step 27



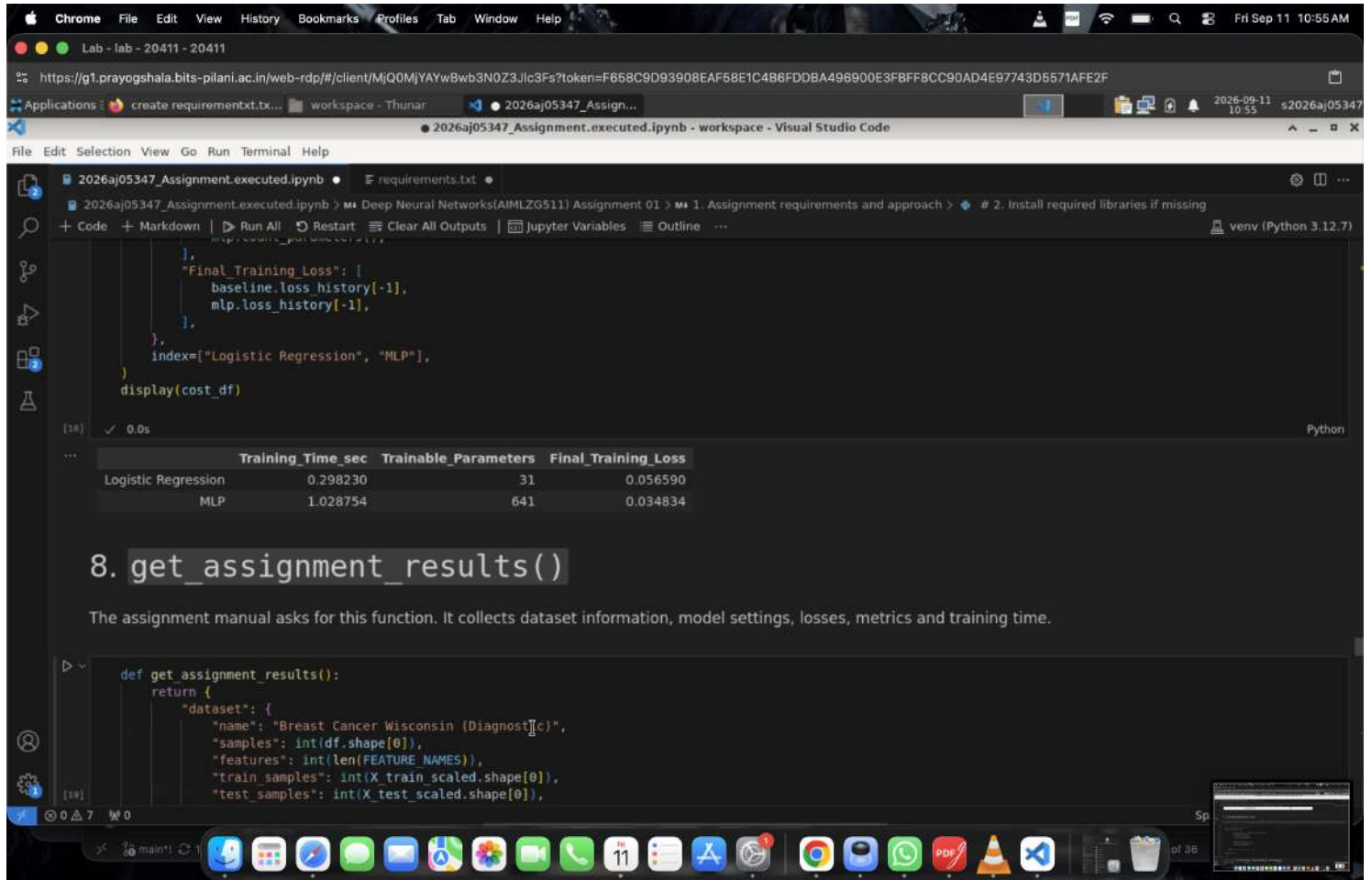
Step 28

The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is part of a workspace named "workspace - Thunar". The browser address bar shows a URL from "g1.prayogshala.bits-pilani.ac.in". The notebook content includes a section titled "7. Computational cost" with a paragraph explaining the assignment's focus on computational-cost difference, training time, and trainable parameters. Below this, a code cell labeled "# 16. Computational comparison" defines a DataFrame named "cost_df" with columns for "Training Time_sec", "Trainable_Parameters", and "Final Training Loss". The DataFrame compares "Logistic Regression" and "MLP". The output of the code cell shows a table with the following data:

	Training_Time_sec	Trainable_Parameters	Final_Training_Loss
Logistic Regression	0.298230	31	0.056590

The bottom of the image shows a macOS dock with various application icons, including Finder, Safari, and VS Code.

Step 29



The screenshot shows a Jupyter Notebook in Visual Studio Code. The notebook is titled "2026aj05347_Assignment.executed.ipynb" and is part of a workspace named "workspace - Thunar". The notebook content includes a code cell that defines a function `get_assignment_results()` and a table showing the results of the function's execution.

```
def get_assignment_results():  
    return {  
        "dataset": {  
            "name": "Breast Cancer Wisconsin (Diagnostic)",  
            "samples": int(df.shape[0]),  
            "features": int(len(FEATURE_NAMES)),  
            "train_samples": int(X_train_scaled.shape[0]),  
            "test_samples": int(X_test_scaled.shape[0]),  
        },  
        "Final_Training_Loss": {  
            "baseline.loss_history[-1]",  
            "mlp.loss_history[-1]",  
        },  
        "index": ["Logistic Regression", "MLP"],  
    }  
display(cost_df)
```

	Training_Time_sec	Trainable_Parameters	Final_Training_Loss
Logistic Regression	0.298230	31	0.056590
MLP	1.028754	641	0.034834

8. `get_assignment_results()`

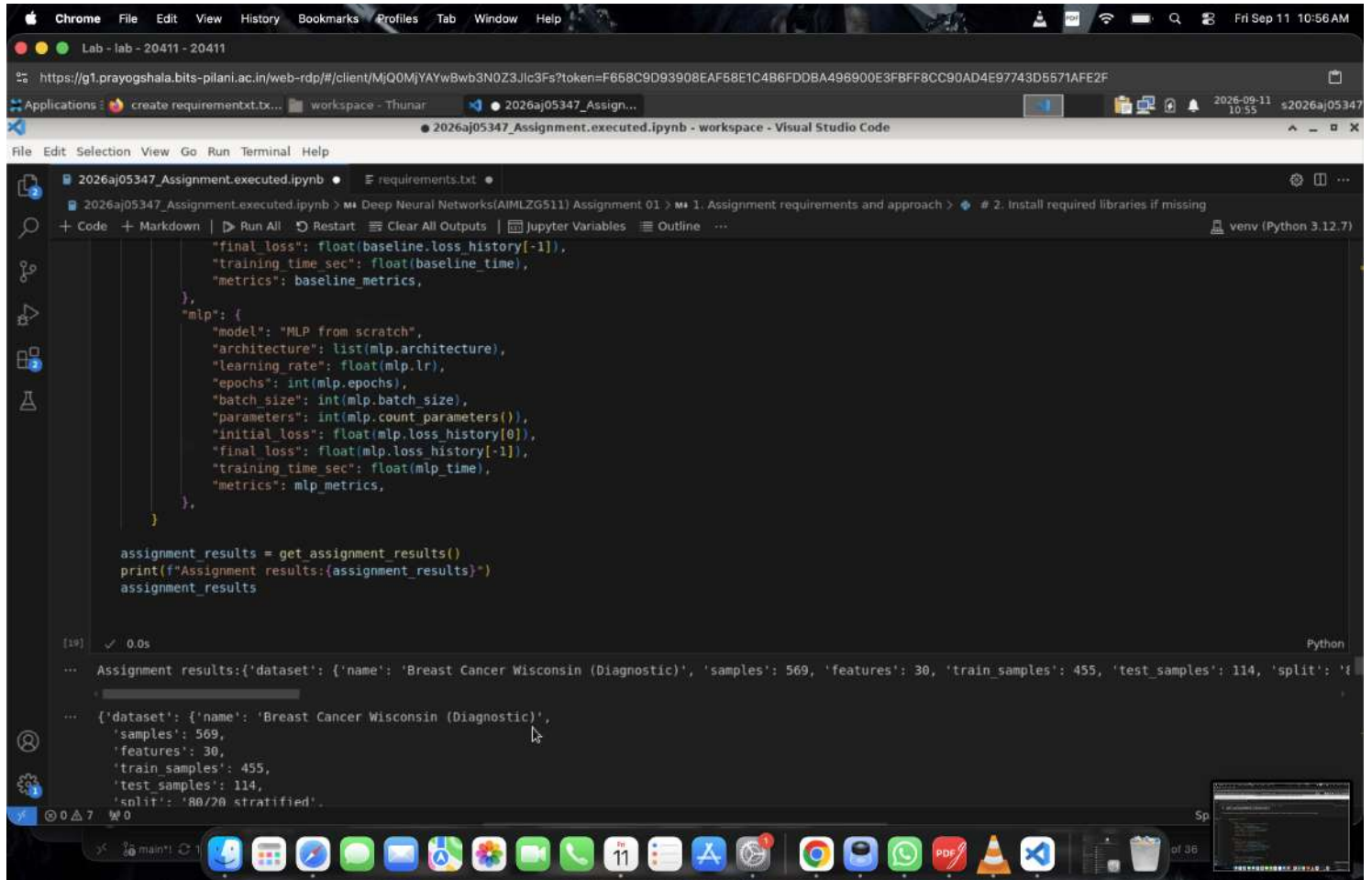
The assignment manual asks for this function. It collects dataset information, model settings, losses, metrics and training time.

```
def get_assignment_results():  
    return {  
        "dataset": {  
            "name": "Breast Cancer Wisconsin (Diagnostic)",  
            "samples": int(df.shape[0]),  
            "features": int(len(FEATURE_NAMES)),  
            "train_samples": int(X_train_scaled.shape[0]),  
            "test_samples": int(X_test_scaled.shape[0]),  
        },  
        "Final_Training_Loss": {  
            "baseline.loss_history[-1]",  
            "mlp.loss_history[-1]",  
        },  
        "index": ["Logistic Regression", "MLP"],  
    }  
display(cost_df)
```

Step 30

```
def get_assignment_results():  
    return {  
        "dataset": {  
            "name": "Breast Cancer Wisconsin (Diagnostic)",  
            "samples": int(df.shape[0]),  
            "features": int(len(FEATURE_NAMES)),  
            "train_samples": int(X_train_scaled.shape[0]),  
            "test_samples": int(X_test_scaled.shape[0]),  
            "split": "80/20 stratified",  
            "primary_metric": "Recall",  
            "positive_class": "Malignant (1)",  
        },  
        "baseline": {  
            "model": "Logistic Regression from scratch",  
            "learning_rate": float(baseline.lr),  
            "iterations": int(baseline.n_iterations),  
            "initial_loss": float(baseline.loss_history[0]),  
            "final_loss": float(baseline.loss_history[-1]),  
            "training_time_sec": float(baseline_time),  
            "metrics": baseline_metrics,  
        },  
        "mlp": {  
            "model": "MLP from scratch",  
            "architecture": list(mlp.architecture),  
            "learning_rate": float(mlp.lr),  
            "epochs": int(mlp.epochs),  
            "batch_size": int(min(batch_size))  
        }  
    }
```

Step 31



The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is running a Python script that defines an MLP model and prints assignment results. The output shows the dataset details and the assignment results.

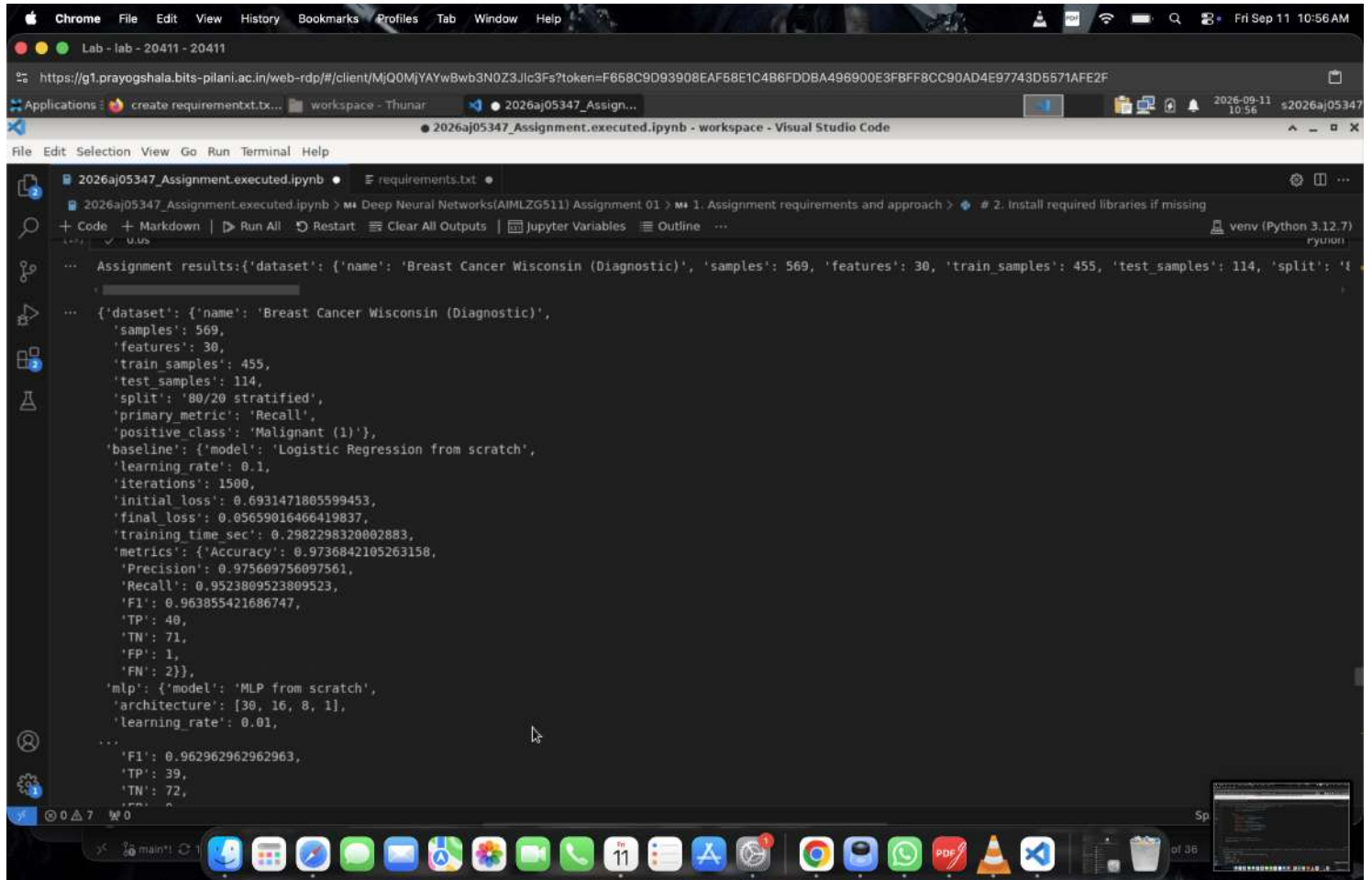
```
...
    "final_loss": float(baseline.loss_history[-1]),
    "training_time_sec": float(baseline_time),
    "metrics": baseline_metrics,
},
"mlp": {
    "model": "MLP from scratch",
    "architecture": list(mlp.architecture),
    "learning_rate": float(mlp.lr),
    "epochs": int(mlp.epochs),
    "batch_size": int(mlp.batch_size),
    "parameters": int(mlp.count_parameters()),
    "initial_loss": float(mlp.loss_history[0]),
    "final_loss": float(mlp.loss_history[-1]),
    "training_time_sec": float(mlp_time),
    "metrics": mlp_metrics,
},
}

assignment_results = get_assignment_results()
print(f"Assignment results:{assignment_results}")
assignment_results
```

Assignment results: {'dataset': {'name': 'Breast Cancer Wisconsin (Diagnostic)', 'samples': 569, 'features': 30, 'train_samples': 455, 'test_samples': 114, 'split': '80/20 stratified'}}

Assignment results: {'dataset': {'name': 'Breast Cancer Wisconsin (Diagnostic)', 'samples': 569, 'features': 30, 'train_samples': 455, 'test_samples': 114, 'split': '80/20 stratified'}}

Step 32



The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is running on a remote server, as indicated by the URL in the browser window. The code cell shows the results of an assignment, including dataset information, baseline model performance, and metrics for a Logistic Regression model.

```
... Assignment results: {'dataset': {'name': 'Breast Cancer Wisconsin (Diagnostic)', 'samples': 569, 'features': 30, 'train_samples': 455, 'test_samples': 114, 'split': '10/20 stratified', 'primary_metric': 'Recall', 'positive_class': 'Malignant (1)'}, 'baseline': {'model': 'Logistic Regression from scratch', 'learning_rate': 0.1, 'iterations': 1500, 'initial_loss': 0.6931471805599453, 'final_loss': 0.05659016466419837, 'training_time_sec': 0.2982298320002883, 'metrics': {'Accuracy': 0.9736842105263158, 'Precision': 0.975609756097561, 'Recall': 0.9523809523809523, 'F1': 0.963855421686747, 'TP': 40, 'TN': 71, 'FP': 1, 'FN': 2}}, 'mlp': {'model': 'MLP from scratch', 'architecture': [30, 16, 8, 1], 'learning_rate': 0.01, ... 'F1': 0.962962962962963, 'TP': 39, 'TN': 72, ...
```

Step 33

The screenshot shows a Jupyter Notebook titled "2026aj05347_Assignment.executed.ipynb" in Visual Studio Code. The notebook is part of a workspace named "2026aj05347_Assign...". The browser address bar shows a URL from "g1.prayogshala.bits-pilani.ac.in". The notebook content includes a code cell with the following output:

```
'Precision': 0.975609756097561,  
'Recall': 0.9523809523809523,  
'F1': 0.963855421686747,  
'TP': 40,  
'TN': 71,  
'FP': 1,  
'FN': 2}},  
'mlp': {'model': 'MLP from scratch',  
'architecture': [30, 16, 8, 1],  
'learning_rate': 0.01,  
...  
'F1': 0.962962962962963,  
'TP': 39,  
'TN': 72,  
'FP': 0,  
'FN': 3}}}
```

Below the output, a message states: "Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)..."

9. Analysis (< 200 words)

The paragraph below is generated from the measured test results and checked for the assignment word-count limit.

```
def build_analysis():  
    baseline_recall = baseline_metrics["Recall"]  
    mlp_recall = mlp_metrics["Recall"]  
  
    if baseline_recall >= mlp_recall:  
        better = "Logistic Regression"  
        diff = baseline_recall - mlp_recall
```

The bottom of the image shows the macOS dock with various application icons.